

Big Data for EDA

Probabilistic Sketches & GPU Acceleration for Chip Analysis  

Wai-Shing Luk 

Fudan University

2026 

Agenda

- 1 🔥 Why approximate? The EDA data explosion
- 2 📊 **Count-Min Sketch** — frequency estimation
- 3 🎯 **HyperLogLog** — cardinality estimation
- 4 ⚡ **GPU acceleration** with Numba CUDA
- 5 📁 **Case study**: switching power analysis
- 6 📊 **Results**: estimates vs. ground truth
- 7 🤖 Big data for **AI-era** chip design
- 8 🚀 Lessons & next steps

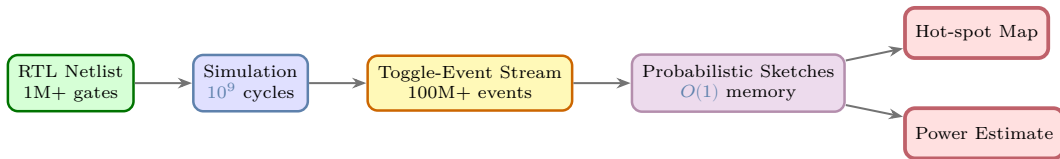
Section 1

🔥 Part 1 — Why Approximate?



The EDA data explosion

- Modern SoCs: **1M+ gates**, **500k nets**, **10 layers** 🧩
- Logic simulation: **10^9 cycles** $\times$ millions of nets 🔄
- Every node toggle is a streaming event $\rightarrow$ exact bookkeeping is $O(n)$ ⚡





Exact vs. approximate

	Exact counting 	Sketch 
Memory	$O(n)$	$O(\frac{1}{\epsilon} \log \frac{1}{\delta})$
Query	exact	probabilistic bound
One-pass stream	× needs storage	✓

Key idea: **trade a tiny, bounded error for a huge memory saving** 

$$\Pr [\hat{c}_x \leq c_x + \epsilon N] \geq 1 - \delta$$

where N is the total number of events. 

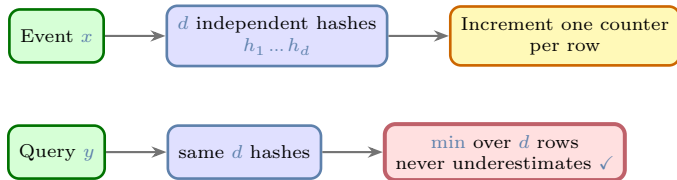
Section 2

📄 Part 2 — Count-Min Sketch



Count-Min Sketch: intuition

Use a **small 2-D array** of counters and d independent hash functions. Increment on every event; query returns the **minimum** across rows.



Collisions only ever **over-count**, so the minimum is a safe estimate. 🎯



Count-Min Sketch: algorithm

Universal hash (Mersenne prime $p = 2^{31} - 1$):

$$h_i(x) = ((a_i x + b_i) \bmod p) \bmod w$$

Update (additive, all rows):

$$\forall i \in [d] : \quad C[i, h_i(x)] += 1$$

Query (minimum = over-estimate):

$$\hat{c}_x = \min_{i \in [d]} C[i, h_i(x)]$$

Guarantee: with probability $\geq 1 - \delta$, $\hat{c}_x \leq c_x + \varepsilon N$



Sizing the sketch

Choose $w = \lceil e/\varepsilon \rceil$ and $d = \lceil \ln(1/\delta) \rceil$.

For $\varepsilon = 0.001$ and $\delta = 0.01$:

$$w = \left\lceil \frac{e}{0.001} \right\rceil = 2719, \quad d = \lceil \ln 100 \rceil = 5$$

$$\text{memory} = w \cdot d \cdot 4 \text{ B} = 2719 \cdot 5 \cdot 4 = 54,380 \text{ B} \approx 53.1 \text{ KB}$$

Additive error budget εN — e.g. $N = 10^8 \Rightarrow \pm 10^5$. 

Demo: switching-activity tracking

CM sketch with only $w = 50$, $d = 3$ tracks 6 nets over **235 toggle events**:

Net	Node ID	Toggles	Estimate
clk_buffer_1	1001	150	✓ 150.0
data_reg_1	2001	75	✓ 75.0
memory_cell	4001	10	✓ 10.0
others	—	0	0.0

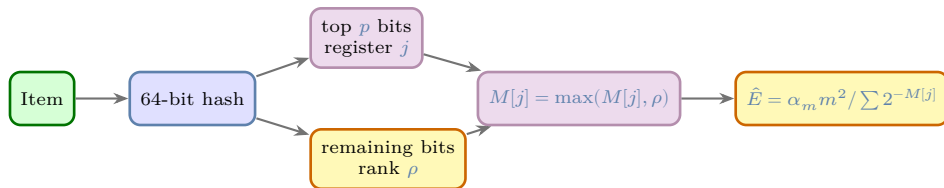
Recovers exact counts **with no collisions** — hot spots are obvious after one pass. 🔥

Section 3

Part 3 — HyperLogLog

 HyperLogLog: intuition

Track the **maximum** rank per register; combine registers with a **harmonic mean**.





HyperLogLog: the math

Registers $m = 2^p$ (precision $p \in [4, 16]$, default $14 \Rightarrow m = 16,384$).

$j = \text{top } p \text{ bits of } h(x)$, $\rho(x) = \# \text{leading zeros of remaining bits}$

$$M[j] = \max(M[j], \rho(x))$$

Estimator (harmonic mean), with $\alpha_m = \frac{0.7213}{1 + 1.079/m}$:

$$\hat{E} = \alpha_m \cdot \frac{m^2}{\sum_{j=1}^m 2^{-M[j]}}$$



Bias corrections & error

Small range (linear counting), when $\hat{E} \leq \frac{5}{2}m$:

$$\hat{E} = m \ln\left(\frac{m}{V}\right), \quad V = \#\{\text{zero registers}\}$$

Large range, when $\hat{E} > \frac{1}{30}2^{64}$:

$$\hat{E} = -2^{64} \ln(1 - \hat{E}/2^{64})$$

Standard error is essentially independent of the cardinality:

$$\sigma \approx \frac{1.04}{\sqrt{m}} \Rightarrow 2.6\% \text{ at } p=10, \quad 0.8\% \text{ at } p=14$$



Memory vs. accuracy: theoretical

$$\sigma \approx \frac{1.04}{\sqrt{m}}, \quad \text{memory} = 4m \text{ bytes (int32 registers)}$$

p	m	Memory	Std. error
8	256	1 KB	6.50%
10	1,024	4 KB	3.25%
12	4,096	16 KB	1.63%
14	16,384	64 KB	0.81%
16	65,536	256 KB	0.41%

256 KB estimates cardinalities of essentially any size.





Demo: measured accuracy

Measured on CPU (SHA-256 hashing), distinct-items benchmark:

p	m	Memory	Measured error
8	256	0.2 KB	0.98%
10	1,024	0.8 KB	2.20%
12	4,096	3.0 KB	2.73%
14	16,384	12 KB	0.46%
16	65,536	48 KB	0.02%

- Merge of two disjoint 5,000-sets $\rightarrow$ est. 10,457 vs. true 10,000 (**4.6% error**) 
- 1M random draws $\rightarrow$ est. 99,470 distinct in **~3 KB** 

HyperLogLog in EDA: application suite

One cardinality primitive, **seven** EDA estimators (map objects $\rightarrow$ typed signatures, then count unique):

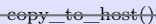
Application	Estimates
DRC	unique violation patterns
Netlist connectivity	unique signal paths · fanout · critical paths
Placement	unique cell types per region
Timing	unique paths · critical paths
Power activity	unique activity patterns · transition counts
Verification	unique states reached · coverage %
Yield	unique defect patterns · failing dies

All with just $m = 2^p$ registers. Coverage, congestion and yield pivot on the same math.

Section 4

⚡ Part 4 — GPU Acceleration

- Registers in global memory



Harmonic-mean estimate

GPU HyperLogLog: kernel design

- One thread per hash; `cuda.grid(1)` gives the thread id 
- Register index from top p bits, rank from the rest

$$j = h(x) \gg (64 - p), \quad \rho = \text{leading-zeros of } h(x) \ll p$$

- **Race-free** update via hardware atomics:

$$M[j] \leftarrow \text{atomicMax}(M[j], \rho)$$

- Hashes precomputed on CPU with **MurmurHash3 x64**, uploaded via `cuda.to_device`





Hashing on the GPU

Two hash families in the demos:

- **xorshift** (`simple_gpu_demo`): $x \leftarrow x \ll 13$; $x \leftarrow x \gg 7$; $x \leftarrow x \ll 17$ — fast, low quality ⚡
- **MurmurHash3 fmix64** (`better_gpu_demo`, `hyper_loglog_gpu_demo`) — well-mixed 🏆

$$k \leftarrow k \oplus (k \gg 33); \quad k \leftarrow k \cdot 0xFF51AFD7ED558CCD; \dots$$

Kernel	Buckets	Target error
b = 14	16,384	$\approx 1\%$
b = 16	65,536	$\approx 0.4\%$

⚡ GPU demos

- `simple_gpu_demo`: **10M** items, $b = 14$ (16,384 buckets) 📊
- `better_gpu_demo`: **10M** items, with linear-counting correction 🎯
- `hyper_loglog_gpu_demo` benchmark: **5M** items, 500k unique, batch sizes 1k $\rightarrow$ 1M 📦
- Memory always $m \times 4$ bytes — 64 KB holds 16,384 registers 💾
Throughput / speed-up depend on the GPU; the demo reports `cpu_time` / `gpu_time` at runtime. ⌚

Section 5

📱 Part 5 — Switching Power Case Study



From toggles to power

Activity factor from the sketch, then the classic dynamic-power formula:

$$\alpha = \frac{n_{\text{toggles}}}{N_{\text{cycles}}}, \quad P = \alpha C V^2 f$$





Demo: full-chip power analysis

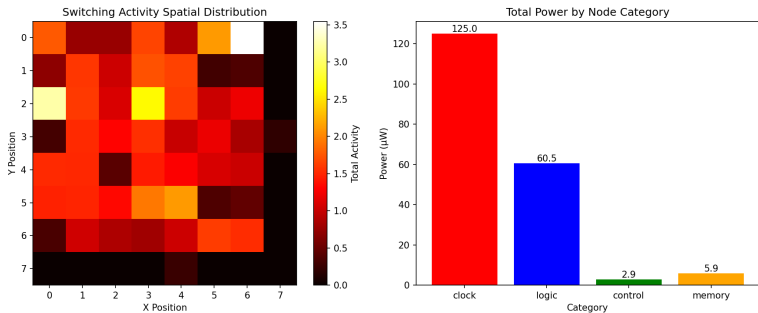
- **500 nodes**, 1,000 cycles, **64,106 toggle events** in **0.69 s** 
- CMS auto-sized to $w = 2719$, $d = 5$ (**53.1 KB**) from $\varepsilon = 0.001$, $\delta = 0.01$

Category	Nodes	C	Total power
 clock	50	5 fF	125.0 μW
 logic	150	2 fF	60.5 μ W
 control	100	1 fF	2.9 μ W
 memory	200	1 fF	5.9 μ W

Hot nodes = the clock tree ($\alpha = 0.5$, 2.5 μ W each) 



Generated: switching power analysis



Power by category and the top power-hungry nodes — from a 53.1 KB sketch. 📊



Temporal activity

Windows of 100 cycles, per-window sketches ($w = 500$, $d = 4$). Two signatures:

Burstiness — spikes vs. steady traffic:

$$B = \frac{\sigma - \mu}{\sigma + \mu}$$

Periodicity — coefficient of variation of inter-event intervals:

$$c_v = \frac{\sigma_{\text{interval}}}{\mu_{\text{interval}}}, \quad \text{periodic if } c_v < 0.3$$

Node	Burstiness	Mean α
periodic_clock	0.00	0.50
bursty_node	0.35	0.16
random_node	0.30	0.17
quiet_node	0.50	0.02

Current envelope

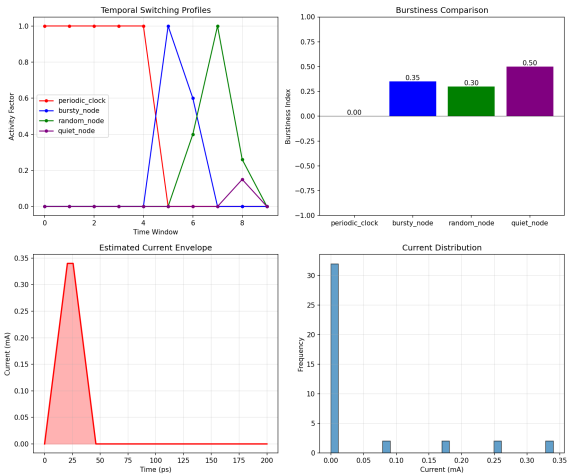
Peak switching current from charge/discharge:

$$I_{\text{peak}} = \frac{CV}{\Delta t}, \quad \Delta t = \text{transition time} = 50 \text{ ps}$$

- Envelope = sum of triangular pulses over hot nodes ▲
- Hot-node filter: activity $\alpha > 0.1$, top-20 nodes  TOP
- Demo: **peak 0.34 mA**, average 0.04 mA 

Sanity-check IR-drop / current-density risks early. 

Generated: temporal & current profile



Per-node activity over time and the estimated current envelope. ⚡

🎵 Pattern-dependent analysis

Different workloads stress different nodes. **Sensitivity index:**

$$SI = \frac{\max_k a_k - \min_k a_k}{\text{mean}_k a_k}$$

- Patterns: Idle, Reset, Max_Load, Worst_Case 🐼
- Toggles: Idle 1,286 · Reset 2,250 · Max_Load 3,121 · **Worst_Case 3,997**
- Most sensitive nodes: **SI = 4.0**, worst pattern = **Max_Load**
- Correlation matrix groups co-active nodes 🔗



Multi-corner (PVT) analysis

Same design across process / voltage / temperature corners:

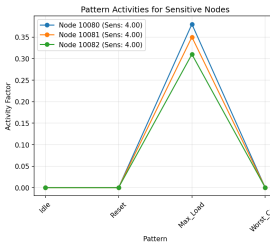
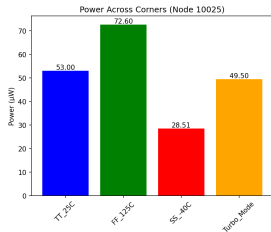
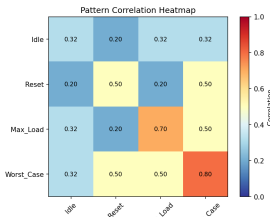
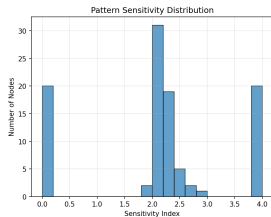
Corner	V	T (°C)	Process
TT_25C	1.0	25	typical
FF_125C	1.1	125	fast
SS_-40C	0.9	-40	slow
Turbo_Mode	1.0	—	boost

Sample node (data path): TT 53.0 μW · FF **72.6** μW · SS 28.5 μW · Turbo 49.5 μW

power variation = 86.6%, worst corner = FF_125C



Generated: pattern & corner analysis



Pattern sensitivity, multi-corner power and co-activity correlation.





Memory $\leftrightarrow$ accuracy trade-off

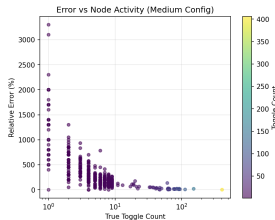
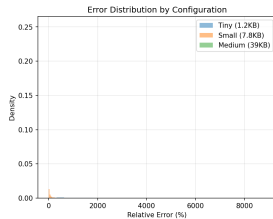
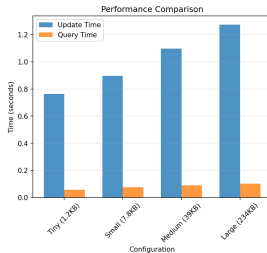
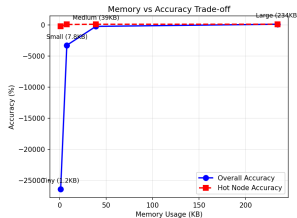
10,000-node power-law workload, **119,266 events**:

Config	Counters	Memory	Avg error	Hot-node error
Tiny	300	1.2 KB	26,538%	323%
Small	2,000	7.8 KB	3,380%	41%
Medium	10,000	39 KB	349%	5.8%
Large	60,000	234 KB	0.16%	0.01%

Punch-line: hot-spot *ranking* is robust even at medium size  — but over-shrinking destroys the tails. 



Generated: memory $\leftrightarrow$ accuracy



Sketch size versus estimation error — the trade-off made precise.



Section 6



Part 6 — Results



EDA application suite: estimates vs. ground truth

Twelve metrics, each from a single-pass HyperLogLog sketch (v0.2.0 run):

Metric	Estimate	Actual	Error
 DRC patterns	300,877	290,919	3.4%
 Signal paths	498,998	500,000	0.2%
 Fanout degrees	49	49	0.0%
 Cell types / region	602	600	0.3%
 Timing paths	99,503	100,000	0.5%
 Critical paths	49,856	50,584	1.4%
 Activity patterns	9,888	9,964	0.8%
 Transition counts	63	63	0.0%
 States covered	256	256	0.0%
 Transitions covered	65,831	65,506	0.5%
 Defect patterns	138	140	1.4%
 Failing-die buckets	636	637	0.2%

All errors stay within HyperLogLog's $\sigma \approx 1.04/\sqrt{m}$ bound. 

Results at a glance

-  **Count-Min Sketch:** recovers 235 toggle events exactly at $w=50$, $d=3$
-  **Full-chip power:** 500 nodes, **64,106 events** $\rightarrow$ **53.1 KB sketch in 0.59 s**; clock tree = 125 μ W of 194 μ W (64%) 
-  **Temporal:** bursty node $B = 0.35$; periodic clock detected (confidence 1.00)
-  **Current envelope:** peak **0.34 mA**, average 0.04 mA
-  **PVT corners:** **86.6%** power variation, worst corner FF_125C
-  **Memory** $\leftrightarrow$ **accuracy:** 39 KB sketch $\rightarrow$ **4–6%** hot-node error; 234 KB $\rightarrow$ **0.01%**
-  **HyperLogLog:** 12 KB $\rightarrow$ **<0.5%** error ($p=14$); **1M items** $\rightarrow$ **3 KB**
-  **EDA suite:** 12 metrics, all $\leq 3.4\%$ error, most $< 1\%$ 

Zero stored elements — every number is computed on the fly. 

Section 7

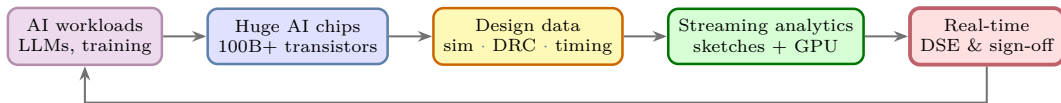


Part 7 — Big Data for AI-Era Chip Design



Chip design in the AI era

- AI workloads demand **enormous silicon**: 100B+ transistors, chiplets, 2 nm 🧩
- **AI-generated RTL** and ML-driven design-space exploration multiply data volume faster than tools scale 📈
- Verification state spaces are **astronomical** — coverage closure dominates schedules 🕒
- Design decisions must now be made **in real time**, inside automated / AI loops 🕒





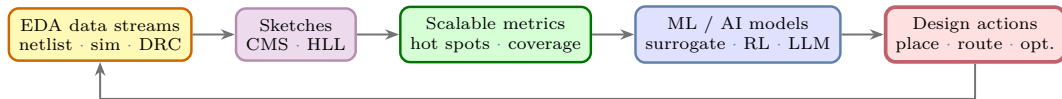
Why sketches + GPU are the enabling layer

-  **Streaming, single pass** — fit simulation, DRC and toggle-event streams without storage
-  **$O(1)$ memory** — analyze billion-gate designs on a workstation, not a cluster
-  **Mergeable** — tile across chiplets, cores and cloud for distributed analysis
-  **GPU-acceleratable** — thousands of parallel hashes/updates for AI-scale data
-  **Bounded error** — statistical guarantees ($\sigma \approx 1.04/\sqrt{m}$) for automated decisions
-  **Foundation for AI-EDA** — fast, scalable metrics feed ML surrogate models and RL placers/routers

Sketches turn **unbounded EDA data into bounded, decision-ready signals** for AI-driven flows. 



From sketches to AI-driven EDA



- Sketches are the **memory-bounded sensing layer** that makes the AI feedback loop affordable ✨
- The same math drives **verification closure, congestion, yield and power** — one toolkit, many decisions 🧰



Lessons learned

- ✓ One pass, bounded memory — sketches fit streaming EDA data
- ✓ Count-Min Sketch → frequencies & hot spots; HyperLogLog → unique counts
- ✓ GPU atomics make register updates race-free and fast ⚙️
- ⚠️ Size accuracy to the *query*: hot-spot ranking is robust, tails are not
- ⚠️ Reproducibility: seed both NumPy **and** the sketch hash RNG 🎲
- ⚠️ Watch for subtle bugs (e.g. merge re-hashing, corner defaults)

Pick the sketch size from the error you can tolerate. 🎯



Q&A

Thanks for listening! 🙏

Wai-Shing Luk 🧑💻 · 2026 📅